

REPUBLIQUE DU CAMEROUN

Paix – Travail – Patrie

MINISTÈRE DES ENSEIGNEMENTS

SUPÉRIEURS

RÉGION DU CENTRE

DÉPARTEMENT DU MFOUNDI

KEYCE INFORMATIQUE ET INTELLI-
GENCE ARTIFICIELLE

E-mail : africa@keyce-informatique.fr

École Supérieure Française

REPUBLIC OF CAMEROON

Peace – Work – Fatherland

MINISTRY OF HIGHER EDUCATION

FAR CENTRAL REGION

MFOUNDI DIVISION

COMPUTER KEYCE AND ARTIFICIAL
INTELLIGENCE

E-mail : africa@keyce-informatique.fr

Thème :

Plateforme collaborative de transport de personnes
et de marchandises inter-localités au Cameroun

(CityShare)

Rapport de projet tutoré

Effectué à Keyce Informatique — Mai 2026

Participants :

EBESSA JOSEPHINE HOT CLARISSE

Matricule : KIA-24-1A-203

MBARGA ERNEST

Matricule : KIA-24-2A-225

MOTCHEYO MENOUNGA GRACE LIANDRA

Matricule :

NGONGANG TCHINCHUT JONATHAN BRYAN

Matricule : KIA-24-2G-313

OTTOU VOUNDI JULIE SAMIRA

Matricule : KIA-24-1U-253

Encadrant :

*FOMEKONG TE-
KATSOP EVARIS*

Filière :

Tronc Commun – B2

Année Acadé-
mique : 2025 – 2026

CityShare — Rapport de projet
tutoré

Keyce Informatique & IA — 2025–2026

*À nos familles, pour leur soutien indéfectible tout au
long de ce parcours académique.*

*À nos encadrants et professeurs de Keyce
Informatique,
pour la qualité de leur enseignement et leur
disponibilité.*

*À tous ceux qui, au Cameroun, peinent chaque jour
à trouver un transport sûr et abordable vers leur
localité.*

Ce travail est le vôtre.

Remerciements

Nous tenons à exprimer notre profonde gratitude à toutes les personnes qui ont contribué à la réalisation de ce projet tutoré.

- **L'établissement Keyce Informatique et Intelligence Artificielle** ainsi que son administration, pour nous avoir fourni le cadre académique, les ressources et les infrastructures nécessaires à notre formation.
- **Notre encadrant, M. FOMEKONG TEKATSOP EVARIS**, pour sa disponibilité, ses conseils avisés et son suivi rigoureux tout au long de ce projet.
- **L'ensemble des enseignants** de la filière Tronc Commun B2, pour la qualité de leur enseignement et les bases solides transmises.
- **Nos familles et proches**, pour leur soutien moral constant durant cette période de travail intensif.

Résumé

CityShare est une plateforme mobile collaborative conçue pour répondre aux défis du transport inter-localités au Cameroun. Elle connecte des conducteurs, des passagers et des expéditeurs de colis sur des trajets communs reliant les grandes villes comme Yaoundé et Douala aux localités secondaires telles que Ntui, Obala et Bafia. Le projet propose un modèle hybride unique associant covoiturage et livraison collaborative (*crowd-shipping*), sécurisé par un système multicouche intégrant vérification d'identité, code [Quick Response \(QR\)](#), bouton SOS et [Dead man's switch](#). Développé avec React Native, Spring Boot et PostgreSQL sans recours à aucune [Application Programming Interface \(API\)](#) payante, CityShare vise à structurer un marché du transport informel encore peu adressé par les solutions numériques existantes.

Mots-clés : covoiturage, crowd-shipping, transport inter-localités, Cameroun, sécurité mobile.

Abstract

CityShare is a collaborative mobile platform designed to address transportation challenges between cities and secondary localities in Cameroon. It connects drivers, passengers, and parcel senders on shared trips linking major cities such as Yaoundé and Douala to smaller towns including Ntui, Obala, and Bafia. The project offers a unique hybrid model combining ride-sharing and crowd-shipping, secured through a multi-layer system featuring identity verification, QR codes, an SOS button, and a dead man's switch. Built with React Native, Spring Boot, and PostgreSQL with no reliance on paid [APIs](#), CityShare aims to bring structure and safety to an informal transport market largely overlooked by existing digital solutions.

Keywords : ride-sharing, crowd-shipping, inter-city transport, Cameroon, mobile security.

Table des figures

1.1	Logo de l'application Yango	4
1.2	Logo des agences de voyage traditionnelles (ex. Buca Voyages)	5
1.3	Logo de l'application BlaBlaCar	5
1.4	Logo de l'application Uber Connect	6
1.5	Logo de l'application Sendy	6
1.6	Logo de l'application Yassir	7
2.1	Diagramme de cas d'utilisation de CityShare	10
2.2	Diagramme de classes simplifié de CityShare	11
2.3	Diagramme d'états-transitions d'un trajet	11
2.4	Diagramme d'activités — réservation d'une place	11
3.1	Écrans basse-fidélité — maquettes Balsamiq de CityShare (14 écrans) .	14
3.2	Écrans haute-fidélité — maquettes Figma de CityShare	15
3.3	Écrans réels de l'application CityShare — 11 fonctionnalités implémentées	17
3.4	Lean Canvas de CityShare	II

Liste des tableaux

- 1.1 Comparaison des solutions existantes avec CityShare 8
- 2.1 Exigences fonctionnelles principales 9
- 2.2 Exigences non fonctionnelles 10
- 3.1 Comparaison des frameworks frontend 13
- 3.2 Comparaison des frameworks backend 13
- 3.3 Comparaison des systèmes de gestion de base de données 14
- 3.4 Principaux *endpoints* de l'API CityShare 18

Liste des abréviations

API Application Programming Interface

B2B Business to Business

CNI Carte Nationale d'Identité

FCM Firebase Cloud Messaging

GPS Global Positioning System

HTTP HyperText Transfer Protocol

HTTPS HyperText Transfer Protocol Secure

JSON JavaScript Object Notation

JWT JSON Web Token

MVP Minimum Viable Product

ORM Object-Relational Mapping

QR Quick Response

REST Representational State Transfer

SQL Structured Query Language

UML Unified Modeling Language

VTC Véhicule de Transport avec Chauffeur

Glossaire

Covoiturage Pratique consistant à partager un véhicule entre plusieurs personnes effectuant le même trajet, permettant de réduire les coûts et l’empreinte carbone

Crowd-shipping Modèle de livraison collaboratif dans lequel des particuliers effectuant un trajet acceptent de transporter des colis pour le compte d’expéditeurs

Dead man’s switch Mécanisme de sécurité déclenchant automatiquement une alerte si un utilisateur n’interagit pas avec l’application dans un délai défini

MVP Version minimale d’un produit comportant les fonctionnalités essentielles permettant de valider les hypothèses métier auprès des premiers utilisateurs

Table des matières

Dédicace	i
Remerciements	i
Résumé	ii
Abstract	iii
Liste des figures	iv
Liste des tableaux	v
Liste des abréviations	vi
Glossaire	vii
Introduction générale	1
1 Concepts et état de l’art	3
1.1 Concepts fondamentaux	3
1.1.1 Application mobile	3
1.1.2 Application Programming Interface REST	3
1.1.3 Covoiturage (<i>ride-sharing</i>)	3
1.1.4 Crowd-shipping (livraison collaborative)	3
1.1.5 Sécurité des systèmes distribués	4
1.2 État de l’art	4
1.2.1 Solutions existantes au Cameroun	4
1.2.2 Solutions existantes à l’international	5
1.3 Bilan comparatif	7
2 Méthodologie et conception	9
2.1 Analyse fonctionnelle	9
2.1.1 Acteurs du système	9
2.1.2 Exigences fonctionnelles	9
2.1.3 Exigences non fonctionnelles	10
2.2 Conception UML	10
2.2.1 Diagramme de cas d’utilisation	10

CityShare — Rapport de projet tutoré

Keyce Informatique & IA — 2025–2026

2.2.2	Diagramme de classes	10
2.2.3	Diagramme d'états-transitions	11
2.2.4	Diagramme d'activités	11
2.3	Architecture du système	11
2.3.1	Architecture logique	11
2.3.2	Architecture sécuritaire	12
2.3.3	Architecture physique	12
2.3.4	Bilan architectural	12
3	Implémentation et résultats	13
3.1	Choix des technologies	13
3.1.1	Frontend : React Native vs Flutter vs Ionic	13
3.1.2	Backend : Spring Boot vs Django vs Node.js	13
3.1.3	Base de données : PostgreSQL vs MySQL vs MongoDB	14
3.2	Résultats obtenus	14
3.2.1	Prototypes	14
3.2.2	Endpoints API	18
3.2.3	État de l'application	18
	Conclusion générale	19
	Références bibliographiques	I
	Annexe A — Lean Canvas de CityShare	II
	Annexe B — Références GitHub	III

Introduction générale

Contexte

Le Cameroun dispose d'un réseau de transport terrestre en pleine mutation, mais les localités secondaires demeurent largement enclavées. Selon les estimations sectorielles, plus de 60 % des trajets entre villes secondaires s'effectuent encore via des véhicules informels, sans garantie tarifaire ni sécuritaire ^[1]. Les solutions numériques de mobilité comme Uber ou Bolt, bien qu'implantées dans les métropoles de Yaoundé et Douala, n'adressent pas les axes inter-localités tels que Yaoundé–Ntui ou Douala–Bafia.

Problématique

Comment concevoir et développer une plateforme numérique fiable, sécurisée et économiquement viable permettant de structurer le transport collaboratif de personnes et de marchandises entre villes et localités secondaires dans le contexte camerounais ?

Motivation

Ce projet naît d'un constat quotidien : des véhicules circulent avec des sièges vides sur ces axes tandis que des voyageurs peinent à trouver un transport sûr et abordable. La convergence du [Covoiturage](#) et de la livraison collaborative sur un même trajet représente une opportunité inédite de valeur partagée pour conducteurs, passagers et commerçants.

Objectifs et plan

L'objectif général est de concevoir, prototyper et implémenter le [MVP](#) de CityShare. Plus spécifiquement, il s'agit :

- (i) d'analyser l'état de l'art des solutions existantes ;
- (ii) de concevoir l'architecture logicielle et les modèles [Unified Modeling Language \(UML\)](#) ;
- (iii) d'implémenter les fonctionnalités essentielles du [Minimum Viable Product \(MVP\)](#) en justifiant chaque choix technologique.

CityShare — Rapport de projet tutoré

Keyce Informatique & IA — 2025–2026

Le rapport comprend trois chapitres : concepts et état de l’art, méthodologie et conception, implémentation et résultats.

Chapitre 1 Concepts et état de l'art

1.1 Concepts fondamentaux

1.1.1 Application mobile

Une application mobile est un logiciel conçu pour s'exécuter sur des appareils portables tels que les smartphones et tablettes. Elle se distingue d'une application web par son accès natif aux capteurs du terminal ([Global Positioning System \(GPS\)](#)), appareil photo, notifications push) et sa capacité à fonctionner en mode déconnecté ^[2]. Dans CityShare, l'application mobile constitue l'interface principale par laquelle conducteurs, passagers et expéditeurs interagissent avec la plateforme.

1.1.2 Application Programming Interface REST

Une API de type [Representational State Transfer \(REST\)](#) est une interface de communication entre composants logiciels reposant sur le protocole [HyperText Transfer Protocol \(HTTP\)](#). Elle expose des ressources via des *endpoints* standardisés (GET, POST, PUT, DELETE) et utilise le format [JavaScript Object Notation \(JSON\)](#) pour l'échange de données ^[3]. CityShare adopte ce paradigme pour toutes les communications entre le *frontend* React Native et le *backend* Spring Boot.

1.1.3 Covoiturage (*ride-sharing*)

Le [Covoiturage](#) désigne le partage d'un véhicule privé entre un conducteur effectuant un trajet pour son propre compte et des passagers souhaitant emprunter le même itinéraire. Ce modèle réduit le coût individuel du transport et le nombre de véhicules sur les routes ^[4]. CityShare étend ce concept aux localités secondaires camerounaises.

1.1.4 Crowd-shipping (livraison collaborative)

Le [Crowd-shipping](#) est un modèle logistique dans lequel des particuliers en déplacement acceptent de transporter des colis pour le compte d'expéditeurs ^[5]. CityShare intègre cette fonctionnalité dans le flux de création de trajet du conducteur, créant une synergie unique entre transport de personnes et de marchandises.

1.1.5 Sécurité des systèmes distribués

Dans un système distribué, la sécurité implique l'authentification des utilisateurs, le chiffrement des communications et le contrôle des accès ^[6]. CityShare implémente un mécanisme d'authentification par **JSON Web Token (JWT)** : chaque requête **API** est accompagnée d'un jeton signé. Ce mécanisme constitue le concept transversal reliant les dimensions technique et métier de la plateforme.

1.2 État de l'art

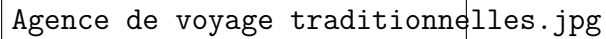
1.2.1 Solutions existantes au Cameroun

 **Yango.** **Pays :** Cameroun (Yaoundé, Douala) | **Année :** 2020 | **Force :** VTC urbain rapide, tarif fixé à l'avance | **Limite :** Zones urbaines uniquement, aucune fonctionnalité inter-localités ni livraison de colis.



Figure 1.1 – *Logo de l'application Yango*

 **Agences de voyage traditionnelles.** **Pays :** Cameroun | **Année :** Antérieur à 2000 | **Force :** Réseau étendu vers les chefs-lieux de département | **Limite :** Aucune réservation numérique, pas de paiement en ligne, horaires fixes sans flexibilité d'arrêts intermédiaires.



Agence de voyage traditionnelles.jpg

Figure 1.2 – *Logo des agences de voyage traditionnelles (ex. Buca Voyages)*

Groupes WhatsApp informels. Pays : Cameroun | **Force** : Covoiturage spontané entre particuliers | **Limite** : Aucun mécanisme de confiance, pas de paiement intégré, aucune garantie sécuritaire pour les participants.

1.2.2 Solutions existantes à l'international

 **BlaBlaCar.** Pays : France (présent dans 22 pays) | **Année** : 2006 | **Force** : Leader mondial du covoiturage longue distance | **Limite** : Non déployé en Afrique subsaharienne, sans livraison de colis.



Figure 1.3 – *Logo de l'application BlaBlaCar*

 **Uber Connect.** Pays : États-Unis | **Année** : 2020 | **Force** : Envoi de colis via des conducteurs Uber existants | **Limite** : Infrastructure d'API payantes, indisponible au Cameroun, ne permet pas le covoiturage de personnes en parallèle.

Uber Freight

Figure 1.4 – *Logo de l'application Uber Connect*

 **Sendy.** Pays : Kenya, Ouganda, Tanzanie | Année : 2014 | Force : Plateforme africaine de livraison Business to Business (B2B) active | Limite : Ne propose pas de covoiturage, pas opérationnelle au Cameroun [8].



Figure 1.5 – *Logo de l'application Sendy*

 **Yassir.** Pays : Algérie | Année : 2017 | Force : Super-application combinant Véhicule de Transport avec Chauffeur (VTC), livraison et services financiers | Limite : Modèle adapté à un marché à fort taux de bancarisation, incompatible avec les réalités camerounaises.



Figure 1.6 – Logo de l'application Yassir

1.3 Bilan comparatif

Conclusion

Au vu de ce tableau, aucune application existante ne remplit l'ensemble des critères essentiels pour le contexte camerounais. CityShare se positionne comme la seule plateforme combinant covoiturage, crowd-shipping, couverture des localités secondaires et sécurité multicouche.

Table 1.1 – Comparaison des solutions existantes avec CityShare

Critère	Yango	BlaBlaCar	Sendy	Yassir	Gp. WA	CityShare
Transport de personnes	Oui	Oui	Non	Oui	Oui	Oui
Livraison de colis	Non	Non	Oui	Oui	Non	Oui
Trajets inter-localités	Non	Non	Non	Non	Oui	Oui
Arrêts intermédiaires flex.	Non	Oui	Non	Non	Non	Oui
Paiement Mobile Money	Non	Non	Non	Non	Non	Oui
Vérification CNI	Non	Non	Non	Oui	Non	Oui
Bouton SOS	Non	Non	Non	Non	Non	Oui
Dead man's switch	Non	Non	Non	Non	Non	Oui
Parrainage communautaire	Non	Non	Non	Non	Oui	Oui
Disponible au Cameroun	Oui	Non	Non	Non	Oui	Oui
Sans API payante	Non	Non	Non	Non	Oui	Oui

Chapitre 2 Méthodologie et conception

2.1 Analyse fonctionnelle

2.1.1 Acteurs du système

CityShare identifie trois acteurs principaux :

- **Conducteur** : publie des trajets, fixe le prix, accepte passagers et colis ;
- **Passager** : recherche et réserve une place ;
- **Expéditeur** : propose un colis à livrer sur un trajet existant.

2.1.2 Exigences fonctionnelles

Table 2.1 – Exigences fonctionnelles principales

ID	Exigence	Priorité
EF01	Création d'un trajet avec départ, arrivée et arrêts	Haute
EF02	Recherche de trajet par ville	Haute
EF03	Proposition d'un colis sur un trajet existant	Haute
EF04	Génération d'un code QR par réservation	Haute
EF05	Alerte SOS avec position GPS	Haute
EF06	Dead man's switch si arrivée non confirmée	Moyenne
EF07	Notifications push via FCM	Moyenne
EF08	Évaluation mutuelle après trajet	Moyenne

2.1.3 Exigences non fonctionnelles

Table 2.2 – *Exigences non fonctionnelles*

Catégorie	Description
Performance	Temps de réponse API ≤ 2 s pour 95 % des requêtes
Sécurité	HTTPS obligatoire ; tokens JWT expirés après 24 h
Disponibilité	99 % de disponibilité hors maintenance
Scalabilité	0 à 5 000 utilisateurs simultanés sans refonte
Utilisabilité	Utilisable sans formation préalable
Maintenabilité	Code versionné sur GitHub (main / dev)

2.2 Conception UML

2.2.1 Diagramme de cas d'utilisation

Le diagramme de la figure 2.1 recense 25 cas d'utilisation répartis en cinq domaines : gestion du compte, gestion des trajets, réservation, gestion des colis et sécurité/urgence.

Figure 2.1 – *Diagramme de cas d'utilisation de CityShare*

2.2.2 Diagramme de classes

Le diagramme de la figure 2.2 modélise neuf entités : Utilisateur, Conducteur, Passager, Expéditeur, Trajet, Réservation, Colis, AlerteSOS et Notation.

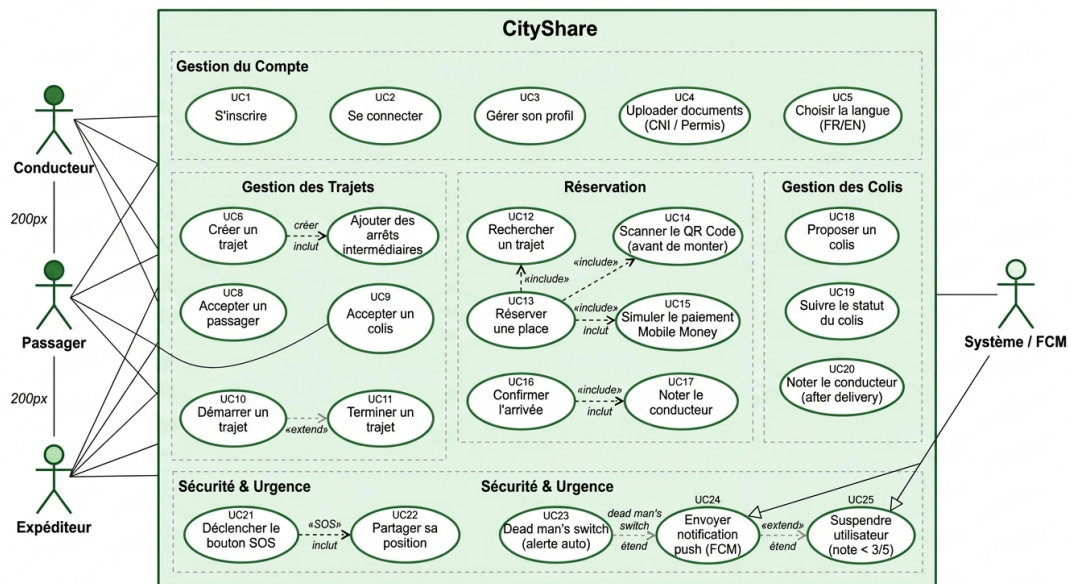


Figure 2.2 – Diagramme de classes simplifié de CityShare

2.2.3 Diagramme d'états-transitions

La figure 2.3 détaille le cycle de vie d'un trajet depuis l'état BROUILLON jusqu'aux états terminaux TERMINÉ et ANNULÉ. Le *Dead man's switch* est activé à l'état EN_COURS.

Figure 2.3 – Diagramme d'états-transitions d'un trajet

2.2.4 Diagramme d'activités

La figure 2.4 détaille le flux de réservation en sept phases : connexion, recherche, réservation, paiement, jour du trajet, trajet en cours et notation.

Figure 2.4 – Diagramme d'activités — réservation d'une place

2.3 Architecture du système

2.3.1 Architecture logique

CityShare adopte une architecture trois-tiers :

- **Couche présentation** : React Native via [REST API](#) ;
- **Couche métier** : Spring Boot, logique de trajet et livraison ;
- **Couche données** : PostgreSQL ([Structured Query Language \(SQL\)](#)).

2.3.2 Architecture sécuritaire

Chaque requête est filtrée par Spring Security qui vérifie le token [JWT](#). Les rôles (CONDUCTEUR, PASSAGER, EXPÉDITEUR) contrôlent l'accès aux ressources. Le protocole [HTTPS](#) est obligatoire.

2.3.3 Architecture physique

Le *backend* est hébergé sur Render. La base PostgreSQL est gérée via Railway. L'application mobile est distribuée depuis les dépôts GitHub.

2.3.4 Bilan architectural

L'architecture retenue garantit une séparation claire des responsabilités, une scalabilité progressive et une maintenance facilitée. Les choix technologiques sont détaillés au chapitre 3.

Chapitre 3 Implémentation et résultats

3.1 Choix des technologies

3.1.1 Frontend : React Native vs Flutter vs Ionic

Table 3.1 – Comparaison des frameworks frontend

Critère	React Native	Flutter	Ionic
Langage	JavaScript/TS	Dart	JavaScript
Performance native	Haute	Très haute	Moyenne
Communauté	Très large	Large	Moyenne
Courbe d'apprentissage	Faible	Moyenne	Faible
Accès GPS/cam/notif.	Oui	Oui	Limité
iOS + Android	Oui	Oui	Oui
Choix retenu		×	×

React Native est retenu pour sa grande communauté, sa faible courbe d'apprentissage et son accès natif aux capteurs. Flutter impose l'apprentissage du langage Dart ; Ionic présente des limitations d'accès aux fonctionnalités mobiles natives.

3.1.2 Backend : Spring Boot vs Django vs Node.js

Table 3.2 – Comparaison des frameworks backend

Critère	Spring Boot	Django	Node.js
Langage	Java	Python	JavaScript
Sécurité (JWT)	Excellente	Bonne	Manuelle
ORM intégré	Spring JPA	Dj. ORM	Sequelize
Performance	Haute	Moyenne	Haute
WebSockets	Oui (STOMP)	Partiel	Oui
Choix retenu		×	×

Spring Boot est retenu pour son module Spring Security natif (JWT et HTTPS sans bibliothèques tierces), son ORM robuste et son support WebSocket via STOMP indispensable pour le suivi en temps réel.

3.1.3 Base de données : PostgreSQL vs MySQL vs MongoDB

Table 3.3 – Comparaison des systèmes de gestion de base de données

Critère	PostgreSQL	MySQL	MongoDB
Modèle	Relationnel	Relationnel	Document
Transactions ACID	Complètes	Complètes	Partielles
Géospatial	Excellent	Limité	Limité
Hébergement gratuit	Render/Railway	Limité	Atlas
Choix retenu		×	×

PostgreSQL est retenu pour son support géospatial natif, ses transactions ACID complètes et sa disponibilité sur les offres gratuites compatibles avec le budget MVP.

3.2 Résultats obtenus

3.2.1 Prototypes

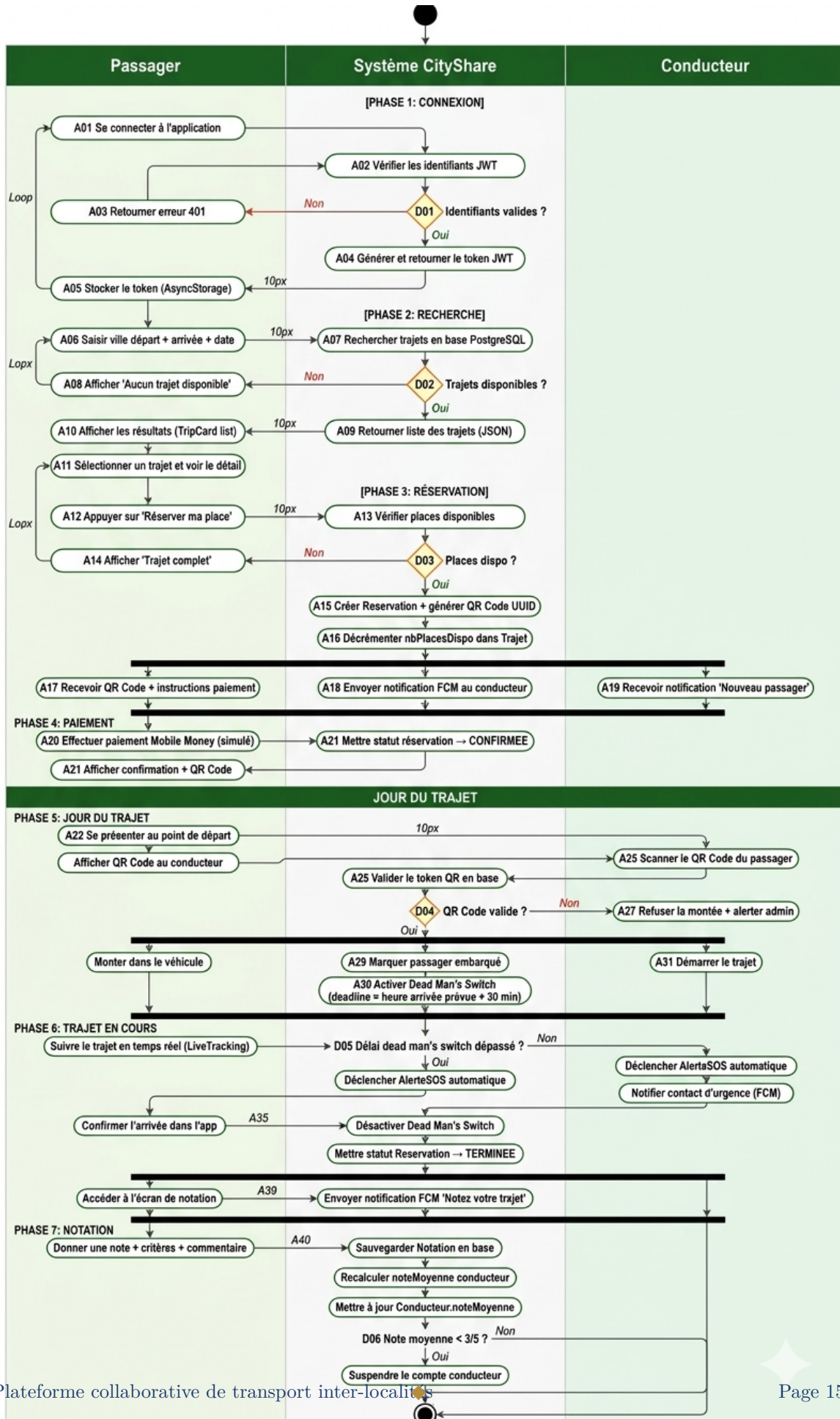
Écrans basse-fidélité (Balsamiq). La figure 3.1 présente les 14 maquettes basse-fidélité réalisées sous Balsamiq, couvrant l’intégralité du parcours utilisateur.

Figure 3.1 – Écrans basse-fidélité — maquettes Balsamiq de CityShare (14 écrans)

Écrans haute-fidélité (Figma). La figure 3.2 présente les écrans haute-fidélité produits sous Figma, intégrant l’identité visuelle CityShare et le parcours complet conducteur/passager.

CityShare — Rapport de projet tutoré

Keyce Informatique & IA — 2025–2026



Application mobile — écrans réels. La figure 3.3 présente les 11 écrans de l'application CityShare telle qu'elle fonctionne sur smartphone : splash screen, accueil passager, dashboard conducteur, suivi de trajet en temps réel sur carte OpenStreet-Map, scanner de code QR, paiement Mobile Money (MTN/Orange), messagerie, alerte SOS, profils passager et conducteur, et formulaire d'envoi de colis.

CityShare — Rapport de projet tutoré

Keyce Informatique & IA — 2025-2026

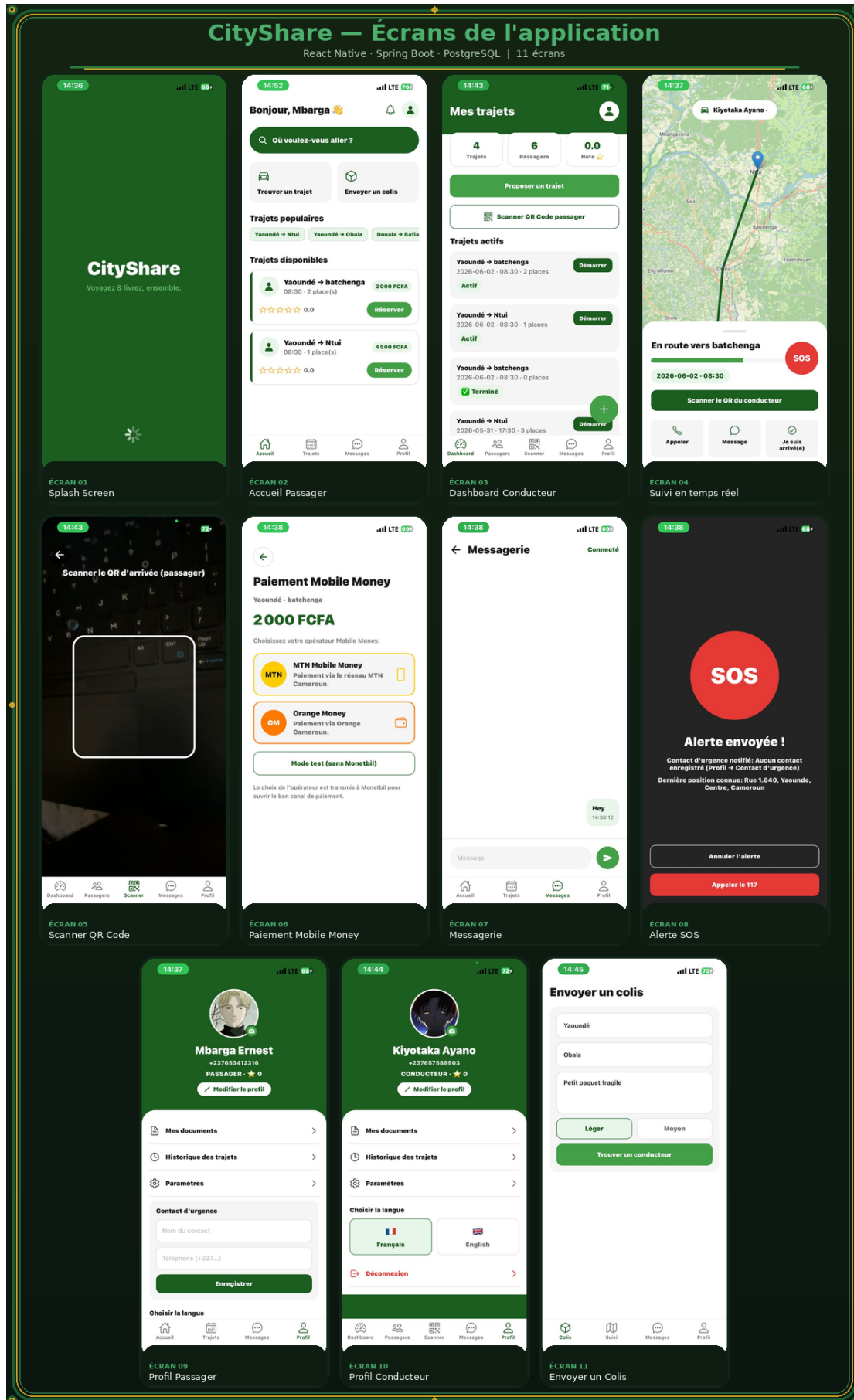


Figure 3.3 – Écrans réels de l'application CityShare — 11 fonctionnalités implémentées

3.2.2 Endpoints API

Table 3.4 – Principaux endpoints de l'API CityShare

Méth.	Endpoint	Description
POST	/api/auth/register	Inscription
POST	/api/auth/login	Connexion + token JWT
POST	/api/trajets	Création d'un trajet
GET	/api/trajets	Recherche de trajets
POST	/api/reservations	Réservation d'une place
GET	/api/reservations/{id}/qr	Code QR
POST	/api/colis	Proposition d'un colis
POST	/api/sos	Alerte SOS
POST	/api/notations	Notation post-trajet

3.2.3 État de l'application

Le *backend* est déployé sur Render. L'application React Native se connecte via [REST](#) authentifié par [JWT](#). La gestion des utilisateurs, la création et la recherche de trajets ainsi que la génération du code [QR](#) sont pleinement opérationnelles à ce stade du [MVP](#).

Conclusion générale

Ce rapport a présenté la conception et le développement de CityShare, une plateforme de transport collaboratif adaptée au contexte camerounais. L'état de l'art confirme l'absence de solution numérique structurée pour les axes Yaoundé–Ntui ou Douala–Bafia. La méthodologie adoptée — analyse fonctionnelle, diagrammes [UML](#), architecture trois-tiers sécurisée par [JWT](#) — a permis de livrer un [MVP](#) fonctionnel sans dépendance à des [API](#) payantes.

Les perspectives à court terme incluent l'intégration d'Orange Money et MTN Mobile Money, l'extension géographique vers de nouveaux corridors et une campagne d'acquisition dans les universités de Yaoundé. À moyen terme, CityShare ambitionne de devenir la référence du transport collaboratif en Afrique centrale.

Références bibliographiques

- [1] Institut National de la Statistique du Cameroun. *Enquête sur les transports et la mobilité urbaine au Cameroun*. INS, Yaoundé, 2022.
- [2] React Native Team. *React Native Documentation*. Meta Platforms, 2024. <https://reactnative.dev/docs/getting-started>
- [3] R. T. Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. Thèse de doctorat, University of California, 2000.
- [4] BlaBlaCar. *BlaBlaCar Annual Report 2023*. Paris, 2023. <https://www.blablacar.com/about-us>
- [5] F. Cîrcu, L. Dragos et M. Popa. “Crowd-shipping logistics : a systematic review.” *Sustainability*, vol. 14, n° 8, 2022.
- [6] W. Stallings. *Network Security Essentials*, 7^e éd. Pearson, 2022.
- [7] Yango Cameroun. *Service Yango au Cameroun*. 2024. https://yango.com/en_cm/
- [8] Sendy Africa. *Sendy Logistics Platform*. Nairobi, 2022. <https://sendy.co/>
- [9] VMware Tanzu. *Spring Boot Reference Documentation 3.x*. 2024. <https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/>
- [10] PostgreSQL Global Development Group. *PostgreSQL 16 Documentation*. 2024. <https://www.postgresql.org/docs/16/>
- [11] A. Cockburn. *Writing Effective Use Cases*. Addison-Wesley, 2001.
- [12] B. Boehm. *Software Engineering Economics*. Prentice Hall, 1981.

Annexe A — Lean Canvas de CityShare

Figure 3.4 – *Lean Canvas de CityShare*

Annexe B — Références GitHub

- **Frontend** : <https://github.com/kiyotaka-sudo/cityshare-frontend>
- **Backend** : <https://github.com/kiyotaka-sudo/cityshare-backend>
- **Branches** : `main` (production) / `dev` (développement)